MeepleAI Operations Manual

> **Generated**: 2026-03-16 | **Stack**: .NET 9 + Next.js 16 + Python AI Services **Target**: Single VPS (Hetzner) with Docker Compose **Audience**: Project owner / sole operator

---

Table of Contents

1. System Overview
2. Docker Environment
3. Secret Management
4. PostgreSQL
5. Qdrant Vector Database
6. Redis Cache
7. API Service (.NET 9)
8. Web Service (Next.js 16)
9. Ollama LLM Runtime
10. Embedding Service
11. Document Processing Services
12. Reranker Service
13. Orchestration Service
14. Monitoring Stack
15. Automation Services
16. Storage Services
17. Incident Response
18. Maintenance & Disaster Recovery

---

1. System Overview

Architecture Diagram (Logical)

```
                        ┌──────────────────────────────────────────┐
                        │              VPS (Hetzner)               │
                        │                                          │
Users ──► :3000 ────────│  web (Next.js 16)                        │
                        │    │                                     │
                        │    ▼                                     │
            :8080 ──────│  api (.NET 9)                            │
                        │    │  ├──► postgres (pgvector)  :5432    │
                        │    │  ├──► qdrant              :6333/:6334 │
                        │    │  ├──► redis               :6379     │
```

```
│    │  └──► orchestration-svc   :8004        │
│    │          ├──► embedding    :8000        │
│    │          ├──► reranker     :8003        │
│    │          ├──► unstructured :8001        │
│    │          └──► smoldocling  :8002        │
│    │                                         │
│    ├──► ollama                  :11434       │
│    └──► minio (S3)             :9000/:9001   │
│                                              │
│  monitoring: prometheus :9090                │
│              grafana    :3001                │
│              alertmanager :9093              │
│              node-exporter :9100             │
│              cadvisor :8080                  │
│                                              │
│  automation: n8n :5678 | mailpit :8025/:1025│
└──────────────────────────────────────────────┘
```

Service Inventory

Service	Image	Container	Profile	Port(s)	CPU	RAM
postgres	pgvector/pgvector:pg16	meepleai-postgres	core	5432	2.0	2G
qdrant	qdrant/qdrant:v1.12.4	meepleai-qdrant	core	6333, 6334	2.0	4G
redis	redis:7.4.1-alpine3.20	meepleai-redis	core	6379	1.0	1G
api	custom build	meepleai-api	core	8080	2.0	4G
web	custom build	meepleai-web	core	3000	1.0	1G
ollama	ollama/ollama:0.5.4	meepleai-ollama	ai	11434	4.0	8G
ollama-pull	curlimages/curl:8.12.1	ollama-pull	ai	-	1.0	512M
embedding-service	custom build	meepleai-embedding	ai	8000	2.0	4G
unstructured-service	custom build	meepleai-unstructured	ai	8001	2.0	2G
smoldocling-service	custom build	meepleai-smoldocling	ai	8002	2.0	4G

reranker-service	custom build	meepleai-reranker	ai	8003	2.0	2G
orchestration-service	custom build	meepleai-orchestrator	ai	8004	2.0	4G
prometheus	prom/prometheus:v3.7.0	meepleai-prometheus	monitoring	9090	1.0	2G
grafana	grafana/grafana:11.4.0	meepleai-grafana	monitoring	3001	1.0	1G
alertmanager	prom/alertmanager:v0.27.0	meepleai-alertmanager	monitoring	9093	0.5	512M
node-exporter	prom/node-exporter:v1.8.2	meepleai-node-exporter	monitoring	9100	0.5	256M
cadvisor	gcr.io/cadvisor/cadvisor:v0.49.1	meepleai-cadvisor	monitoring	8080	0.5	512M
n8n	n8nio/n8n:1.114.4	meepleai-n8n	automation	5678	1.0	1G
mailpit	axllent/mailpit:v1.22	mailpit	automation	8025, 1025	0.5	128M
minio	minio/minio:RELEASE.2024-11-07T00-52-20Z	meepleai-minio	storage	9000, 9001	0.5	512M
minio-init	minio/mc:RELEASE.2024-11-17T19-35-25Z	meepleai-minio-init	storage	-	-	-

Named Volumes (13)

Category	Volume Names
Database & Cache	`postgres_data`, `qdrant_data`
ML Model Caches	`ollama_data`, `smoldocling_models`, `reranker_models`
Temp Processing	`unstructured_temp`, `smoldocling_temp`
Application Data	`pdf_uploads`, `minio_data`
Observability	`prometheus_data`, `grafana_data`, `alertmanager_data`, `mailpit_data`

Additional in overrides: `postgres_dev_data` (dev), `traefik_certs` (traefik), 5 prod volumes, 5 staging volumes.

Dependency Graph (Startup Order)

```
postgres (healthy) ──┬──► api (healthy) ──► web
redis (healthy) ─────┘        │
qdrant (started) ─────────────┘
                              ├──► n8n
ollama (healthy) ──► ollama-pull
embedding-service (started) ──┬──► orchestration-service
reranker-service (started) ───┘
prometheus ──► grafana
minio (healthy) ──► minio-init
```

Resource Budget Summary

Profile	Services	Total RAM	Total CPU	Use Case
minimal (core)	6	~7 GB	~4	Backend development
dev	12	~15 GB	~8	Full dev + monitoring
ai	11	~25 GB	~12	AI/ML feature development
automation	8	~10 GB	~8	n8n workflow dev
monitoring	13	~18 GB	~8	Monitoring stack
full	21	~41 GB	~27	Production (all services)

Scaling Tiers (Hetzner Cloud)

Plan	vCPU	RAM	Price/mo	Suitable For
CPX31	4	8 GB	EUR 15.59	Beta / core only
CPX41	8	16 GB	EUR 29.52	Small production
CPX51	16	32 GB	EUR 65.18	Medium production (AI enabled)

---

2. Docker Environment

Compose File Structure

```
infra/
├── docker-compose.yml          # Base (no ports, no env_file — just images, volumes, networks
├── compose.dev.yml             # Dev overrides (ports, env_file, labels)
├── compose.staging.yml         # Staging overrides
```

```
├── compose.prod.yml            # Production overrides
├── compose.traefik.yml         # Traefik reverse proxy overlay
├── compose.integration.yml     # CI integration tests
├── compose.override.yml.example # Local developer overrides template
```

The base `docker-compose.yml` is **environment-neutral**: ports, `env_file`, and `environment` blocks live in per-environment overrides.

Starting Services

```
# Core services only (development)
cd infra
docker compose -f docker-compose.yml -f compose.dev.yml up -d

# Core + AI services
docker compose -f docker-compose.yml -f compose.dev.yml --profile ai up -d

# Core + Monitoring
docker compose -f docker-compose.yml -f compose.dev.yml --profile monitoring up -d

# Core + Automation
docker compose -f docker-compose.yml -f compose.dev.yml --profile automation up -d

# Core + Storage (MinIO)
docker compose -f docker-compose.yml -f compose.dev.yml --profile storage up -d

# Everything
docker compose -f docker-compose.yml -f compose.dev.yml \
  --profile ai --profile monitoring --profile automation --profile storage up -d

# Helper scripts (simpler)
pwsh infra/start-minimal.ps1    # Core only
pwsh infra/start-dev.ps1        # Core + monitoring
bash infra/start-ai.sh          # Core + AI
bash infra/start-automation.sh  # Core + automation
```

Stopping Services

```
# Stop all running services
cd infra && docker compose down

# Stop and remove volumes (DATA LOSS)
docker compose down -v

# Stop specific profile
docker compose --profile ai stop
```

```
# PowerShell helper
pwsh infra/stop.ps1
```

Checking Status

```
# Service status
docker compose ps

# Real-time resource usage
docker stats --no-stream

# Disk usage by Docker
docker system df

# Detailed disk usage
docker system df -v
```

Network Configuration

All services share a single bridge network: `meepleai`. Services communicate by container name (e.g., `postgres`, `redis`, `qdrant`).

```
# Inspect network
docker network inspect meepleai

# Test connectivity between containers
docker exec meepleai-api ping -c 2 meepleai-postgres
```

Docker Maintenance

```
# Remove stopped containers, unused networks, dangling images
docker system prune -f

# Aggressive cleanup (removes ALL unused images)
docker system prune -a -f

# Remove specific volume
docker volume rm meepleai_postgres_data

# Rebuild a specific service
docker compose build --no-cache api
```

```
# Pull latest images (for non-custom-built services)
docker compose pull postgres qdrant redis
```

Environment Override for Local Development

Copy the example override and customize:

```
cd infra
cp compose.override.yml.example compose.override.yml
# Edit compose.override.yml to add local ports, environment variables, etc.
```

Docker Compose automatically picks up `compose.override.yml` when present.

---

3. Secret Management

Overview

MeepleAI uses **file-based secrets** (`.secret` files in `infra/secrets/`). Each file contains environment variables in `KEY=VALUE` format. Secrets are loaded via `load-secrets-env.sh` at container startup.

Secret Tiers

Tier	Files	Behavior on Missing
CRITICAL (6)	`admin.secret`, `database.secret`, `embedding-service.secret`, `jwt.secret`, `qdrant.secret`, `redis.secret`	Blocks startup
IMPORTANT (3)	`bgg.secret`, `openrouter.secret`, `unstructured-service.secret`	Logs warning
OPTIONAL (7)	`email.secret`, `monitoring.secret`, `oauth.secret`, `reranker-service.secret`, `smoldocling-service.secret`, `storage.secret`, `traefik.secret`	Silent fallback

Initial Setup

```
cd infra/secrets
pwsh setup-secrets.ps1 -SaveGenerated
```

This script:

1. Checks for existing `.secret` files
2. Generates secure random values for missing files
3. Saves generated files with `-SaveGenerated` flag
4. Validates all critical secrets are present

Secret File Format

Each `.secret` file is a flat `KEY=VALUE` file:

```
# infra/secrets/database.secret
POSTGRES_USER=meepleai
POSTGRES_PASSWORD=<generated-secure-value>
POSTGRES_DB=meepleai
```

Rotation Procedure

```
# 1. Generate new secret value
pwsh -c "[System.Convert]::ToBase64String((1..32 | % { Get-Random -Max 256 }) -as [byte[]])

# 2. Update the .secret file
nano infra/secrets/jwt.secret

# 3. Restart affected services
docker compose restart api

# 4. Verify service health
docker compose ps
```

Rotation Schedule

Secret	Rotation Period	Notes
JWT signing key	90 days	Invalidates all active sessions
Database password	180 days	Requires DB ALTER ROLE
Admin password	90 days	Update in `admin.secret`
API keys (BGG, OpenRouter)	On compromise	Third-party service keys
Redis password	180 days	Restart Redis + all clients
Qdrant API key	180 days	Restart Qdrant + API
All others	365 days	Low-risk secrets

Security Rules

- **NEVER** commit `.secret` files to git (`.gitignore` enforced)
- **NEVER** use development secrets in production
- **ALWAYS** run `setup-secrets.ps1 -SaveGenerated` on fresh deployment
- **ALWAYS** use the `load-secrets-env.sh` entrypoint pattern (not Docker `environment:` blocks)
- Back up secrets separately from database backups
- Store a copy of production secrets in an encrypted vault (e.g., Bitwarden)

---

4. PostgreSQL

Overview

Property	Value
Image	`pgvector/pgvector:pg16`
Container	`meepleai-postgres`
Port	5432
Volume	`postgres_data` → `/var/lib/postgresql/data`
Init Script	`infra/init/postgres-init.sql` (mounted at startup)
CPU / RAM Limit	2.0 / 2G
CPU / RAM Reserved	1.0 / 1G
Shared Memory	`shm_size: 1g`
Health Check	`pg_isready -U $POSTGRES_USER -d $POSTGRES_DB` (5s interval, 3s timeout, 10 retries)
Extensions	pgvector (vector similarity search)

Daily Operations

```
# Check if PostgreSQL is running and accepting connections
docker exec meepleai-postgres pg_isready -U meepleai

# Check active connections
docker exec meepleai-postgres psql -U meepleai -c \
  "SELECT count(*) FROM pg_stat_activity;"

# Check database size
```

```
docker exec meepleai-postgres psql -U meepleai -c \
  "SELECT pg_size_pretty(pg_database_size('meepleai'));"

# List largest tables
docker exec meepleai-postgres psql -U meepleai -c \
  "SELECT relname, pg_size_pretty(pg_total_relation_size(relid))
   FROM pg_catalog.pg_statio_user_tables
   ORDER BY pg_total_relation_size(relid) DESC
   LIMIT 10;"

# Check for long-running queries (>30 seconds)
docker exec meepleai-postgres psql -U meepleai -c \
  "SELECT pid, now()-query_start AS duration, state, query
   FROM pg_stat_activity
   WHERE state='active' AND now()-query_start > interval '30 seconds';"

# Kill long-running queries
docker exec meepleai-postgres psql -U meepleai -c \
  "SELECT pg_cancel_backend(pid)
   FROM pg_stat_activity
   WHERE state='active' AND now()-query_start > interval '30 seconds';"

# View replication and WAL status
docker exec meepleai-postgres psql -U meepleai -c \
  "SELECT pg_current_wal_lsn(), pg_wal_lsn_diff(pg_current_wal_lsn(), '0/0') AS wal_bytes;"

# Check dead tuples (bloat indicator)
docker exec meepleai-postgres psql -U meepleai -c \
  "SELECT relname, n_dead_tup, n_live_tup, last_vacuum, last_autovacuum
   FROM pg_stat_user_tables
   WHERE n_dead_tup > 1000
   ORDER BY n_dead_tup DESC;"

# View container logs
docker logs meepleai-postgres --tail=50
```

Backup & Restore

Backup (Full Dump)

```
# Full cluster backup (all databases)
docker exec meepleai-postgres pg_dumpall -U meepleai | gzip > "backup_$(date +%Y%m%d_%H%M%S

# Single database backup
docker exec meepleai-postgres pg_dump -U meepleai meepleai | gzip > "meepleai_$(date +%Y%m%
```

```
# Custom format (supports parallel restore)
docker exec meepleai-postgres pg_dump -U meepleai -Fc meepleai > "meepleai_$(date +%Y%m%d_%
```

Automated daily backup (add to crontab or Task Scheduler):

```
# crontab -e
0 3 * * * docker exec meepleai-postgres pg_dumpall -U meepleai | gzip > /backups/postgres/d
```

Backup policy: Daily at 03:00, 7-day retention.

Restore

```
# Stop the API to prevent writes during restore
docker compose stop api web

# Restore from full dump
gunzip -c backup_20260316.sql.gz | docker exec -i meepleai-postgres psql -U meepleai

# Restore from custom format
docker exec -i meepleai-postgres pg_restore -U meepleai -d meepleai < meepleai_20260316.dum

# Restart services
docker compose start api web
```

Point-in-Time Verification

```
# After restore, verify row counts on key tables
docker exec meepleai-postgres psql -U meepleai -c \
  "SELECT 'users' AS tbl, count(*) FROM \"Administration\".\"Users\"
   UNION ALL
   SELECT 'games', count(*) FROM \"GameManagement\".\"Games\";"
```

Scaling & Performance Tuning

Tuning Tiers

Tier	Users	shared_buffers	work_mem	effective_cache_size	Peak RAM
Small	~100	512 MB	16 MB	1.5 GB	~2 GB
Medium	~1,000	768 MB	16 MB	2 GB	~3 GB

Large	~10,000	1,280 MB	16 MB	3 GB	~5 GB

Apply tuning by adding a `postgresql.conf` override:

```
# Create custom config
cat > infra/postgres/custom.conf << 'EOF'
# Connection Management
max_connections = 100
idle_in_transaction_session_timeout = '30s'

# Memory (adjust per tier above)
shared_buffers = 512MB
work_mem = 16MB
maintenance_work_mem = 128MB
effective_cache_size = 1536MB

# WAL
wal_buffers = 16MB
checkpoint_completion_target = 0.9

# Autovacuum (keep aggressive for MVCC health)
autovacuum_max_workers = 3
autovacuum_naptime = 30s
autovacuum_vacuum_threshold = 50
autovacuum_analyze_threshold = 50

# Logging
log_min_duration_statement = 1000
log_checkpoints = on
log_lock_waits = on
EOF

# Mount in docker-compose override
# volumes:
#   - ./postgres/custom.conf:/etc/postgresql/custom.conf:ro
# command: postgres -c config_file=/etc/postgresql/custom.conf
```

Connection Pool Monitoring

```
# Check connection usage vs max
docker exec meepleai-postgres psql -U meepleai -c \
  "SELECT count(*) AS active, max_conn
   FROM pg_stat_activity, (SELECT setting::int AS max_conn FROM pg_settings WHERE name='max
   GROUP BY max_conn;"
```

Upgrade & Migration

Minor Version Upgrade (16.x → 16.y)

```
# 1. Backup
docker exec meepleai-postgres pg_dumpall -U meepleai | gzip > pre_upgrade.sql.gz

# 2. Pull new image
docker compose pull postgres

# 3. Recreate container
docker compose up -d postgres

# 4. Verify
docker exec meepleai-postgres psql -U meepleai -c "SELECT version();"
```

Major Version Upgrade (16 → 17)

```
# 1. Full backup
docker exec meepleai-postgres pg_dumpall -U meepleai | gzip > pre_major_upgrade.sql.gz

# 2. Stop all services
docker compose down

# 3. Remove old volume
docker volume rm meepleai_postgres_data

# 4. Update image in docker-compose.yml
# image: pgvector/pgvector:pg17

# 5. Start fresh PostgreSQL
docker compose up -d postgres

# 6. Restore
gunzip -c pre_major_upgrade.sql.gz | docker exec -i meepleai-postgres psql -U meepleai

# 7. Run EF Core migrations
cd apps/api/src/Api && dotnet ef database update

# 8. Restart all services
cd infra && docker compose up -d
```

EF Core Migrations

```
cd apps/api/src/Api

# Create new migration
dotnet ef migrations add DescriptiveName

# Apply pending migrations
dotnet ef database update

# List migrations
dotnet ef migrations list

# Rollback to specific migration
dotnet ef database update PreviousMigrationName

# Generate SQL script (for review)
dotnet ef migrations script PreviousMigration NewMigration
```

Troubleshooting

Symptom	Diagnosis	Fix
Connection refused	`docker compose ps postgres`	Restart: `docker compose restart postgres`
Too many connections	Check `pg_stat_activity` count	Kill idle sessions, increase `max_connections`
Slow queries	`log_min_duration_statement = 500`	Add indexes, run `ANALYZE`
Table bloat / dead tuples	Check `n_dead_tup`	Run `VACUUM FULL ANALYZE;`
Disk full	`docker system df`	Prune old backups, run VACUUM
WAL accumulation	Check `pg_wal_lsn_diff`	Checkpoint: `SELECT pg_switch_wal();`
"FATAL: role does not exist"	Secret mismatch	Verify `database.secret` matches init script
pgvector extension missing	`\dx` shows no vector	`CREATE EXTENSION IF NOT EXISTS vector;`

```
# Emergency: force immediate VACUUM on all tables
docker exec meepleai-postgres psql -U meepleai -c "VACUUM FULL ANALYZE;"
```

```
# Emergency: terminate all connections except own
docker exec meepleai-postgres psql -U meepleai -c \
  "SELECT pg_terminate_backend(pid) FROM pg_stat_activity
   WHERE pid <> pg_backend_pid() AND datname='meepleai';"
```

Disaster Recovery

- **RTO**: 30-60 minutes (restore from dump)
- **RPO**: 24 hours (daily backup schedule)
- **Procedure**: See Section 18 - Disaster Recovery

Cost Notes

- PostgreSQL is open source, zero license cost
- Storage: ~1 GB per 100K rows (varies by schema)
- pgvector adds ~4 KB per vector (1024 dimensions)
- Backup storage: ~30% of DB size compressed

---

5. Qdrant Vector Database

Overview

Property	Value
Image	`qdrant/qdrant:v1.12.4`
Container	`meepleai-qdrant`
Ports	6333 (HTTP API), 6334 (gRPC)
Volume	`qdrant_data` → `/qdrant/storage`
CPU / RAM Limit	2.0 / 4G
CPU / RAM Reserved	1.0 / 2G
Health Check	TCP check on `/readyz` (10s interval, 5s timeout, 10 retries, 10s start period)
Collection	`meepleai_documents`
Vector Dimension	1024
HNSW Config	M=16 (default)

Daily Operations

```
# Health check
curl http://localhost:6333/healthz

# Check cluster status
curl http://localhost:6333/cluster -H "api-key: $QDRANT_API_KEY"

# List collections
curl http://localhost:6333/collections -H "api-key: $QDRANT_API_KEY"

# Collection details (point count, config)
curl http://localhost:6333/collections/meepleai_documents \
  -H "api-key: $QDRANT_API_KEY"

# Collection size and segment info
curl http://localhost:6333/collections/meepleai_documents/cluster \
  -H "api-key: $QDRANT_API_KEY"

# Count points in collection
curl -X POST http://localhost:6333/collections/meepleai_documents/points/count \
  -H "api-key: $QDRANT_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"exact": true}'

# View container logs
docker logs meepleai-qdrant --tail=50
```

Backup & Restore

Backup (Snapshots)

```
# Create snapshot of a collection
curl -X POST "http://localhost:6333/collections/meepleai_documents/snapshots" \
  -H "api-key: $QDRANT_API_KEY"

# List snapshots
curl "http://localhost:6333/collections/meepleai_documents/snapshots" \
  -H "api-key: $QDRANT_API_KEY"

# Download snapshot
curl "http://localhost:6333/collections/meepleai_documents/snapshots/<snapshot_name>" \
  -H "api-key: $QDRANT_API_KEY" \
  --output qdrant_backup.snapshot

# Full storage snapshot (all collections)
```

```
curl -X POST "http://localhost:6333/snapshots" \
  -H "api-key: $QDRANT_API_KEY"
```

Restore

```
# Stop Qdrant
docker compose stop qdrant

# Upload snapshot to restore
curl -X POST "http://localhost:6333/collections/meepleai_documents/snapshots/upload" \
  -H "api-key: $QDRANT_API_KEY" \
  -H "Content-Type: multipart/form-data" \
  -F "snapshot=@qdrant_backup.snapshot"

# Alternatively: restore from full snapshot
curl -X POST "http://localhost:6333/snapshots/upload" \
  -H "api-key: $QDRANT_API_KEY" \
  -F "snapshot=@full_backup.snapshot"

# Start Qdrant
docker compose start qdrant
```

Backup policy: Weekly snapshots, retained for 30 days. Vectors can be regenerated from source documents via the embedding pipeline.

Scaling & Performance Tuning

RAM Requirements Per User

User Count	Vectors (est.)	RAM Required	Strategy
< 1,000	~156K	~400 MB	In-memory (default)
1,000 - 10,000	~1.56M	~4 GB	In-memory, 4G limit sufficient
10,000 - 50,000	~7.8M	~6 GB+	Enable mmap or quantization
> 50,000	> 7.8M	> 6 GB	Sharding required

Calculation: ~156 vectors per user x ~2.5 KB per vector (1024-dim float32) = ~390 KB/user.

Enable Memory-Mapped Storage (for large datasets)

```
curl -X PATCH "http://localhost:6333/collections/meepleai_documents" \
  -H "api-key: $QDRANT_API_KEY" \
  -H "Content-Type: application/json" \
```

```
  -d '{
    "optimizers_config": {
      "memmap_threshold": 20000
    }
  }'
```

Enable Scalar Quantization (reduce RAM ~4x)

```
curl -X PATCH "http://localhost:6333/collections/meepleai_documents" \
  -H "api-key: $QDRANT_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "quantization_config": {
      "scalar": {
        "type": "int8",
        "always_ram": true
      }
    }
  }'
```

Upgrade & Migration

```
# 1. Create snapshot before upgrade
curl -X POST "http://localhost:6333/snapshots" -H "api-key: $QDRANT_API_KEY"

# 2. Update image tag in docker-compose.yml
# image: qdrant/qdrant:v1.13.0

# 3. Pull and recreate
docker compose pull qdrant
docker compose up -d qdrant

# 4. Verify
curl http://localhost:6333/healthz
curl http://localhost:6333/collections/meepleai_documents -H "api-key: $QDRANT_API_KEY"
```

Troubleshooting

Symptom	Diagnosis	Fix
Health check fails	`curl http://localhost:6333/healthz`	Check logs: `docker logs meepleai-qdrant`
High memory usage	Collection growing beyond limit	Enable mmap or quantization

Slow search	Large collection, no optimization	Rebuild HNSW index, enable quantization
Collection not found	Wrong collection name	`curl localhost:6333/collections` to list
API key rejected	Secret mismatch	Verify `qdrant.secret`
gRPC connection refused	Port 6334 not exposed	Check compose override exposes 6334

Disaster Recovery

- **RTO**: 15-30 minutes (restore from snapshot)
- **RPO**: 7 days (weekly snapshots) — vectors are re-generatable from source docs
- **Nuclear option**: Delete collection and re-embed all documents through the pipeline

Cost Notes

- Qdrant is open source, zero license cost
- RAM-dominant cost: ~2.5 KB per 1024-dim vector
- Quantization reduces storage 4x at minor quality loss
- Snapshots: ~60-80% of live storage size

---

6. Redis Cache

Overview

Property	Value
Image	`redis:7.4.1-alpine3.20`
Container	`meepleai-redis`
Port	6379
Volume	Bind mount: `./redis/redis.conf` → `/etc/redis/redis.conf:ro`
CPU / RAM Limit	1.0 / 1G
CPU / RAM Reserved	0.5 / 512M
Health Check	`redis-cli ping` (10s interval, 3s timeout, 5 retries)
Auth	Password from `redis.secret` via `REDIS_PASSWORD` env var

Redis Configuration

The `redis.conf` at `infra/redis/redis.conf` sets:

```
appendonly yes
maxmemory 768mb
maxmemory-policy allkeys-lru
```

- **appendonly yes**: AOF persistence — every write is logged for durability
- **maxmemory 768mb**: Hard cap within the 1G container limit (leaves room for overhead)
- **maxmemory-policy allkeys-lru**: When full, evicts least-recently-used keys across all keyspaces

Daily Operations

```
# Ping (authentication required)
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" ping

# Server info summary
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" INFO stats

# Memory usage
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" INFO memory

# Connected clients
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" INFO clients

# Find large keys
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" --bigkeys

# Key count per database
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" INFO keyspace

# Monitor live commands (use briefly — high overhead)
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" MONITOR

# Check eviction stats
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" INFO stats | grep evicted

# View container logs
docker logs meepleai-redis --tail=50
```

Backup & Restore

Backup

```
# Trigger background save
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" BGSAVE
```

```
# Wait for save to complete
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" LASTSAVE

# Copy dump file out
docker cp meepleai-redis:/data/dump.rdb "redis_backup_$(date +%Y%m%d).rdb"

# Copy AOF file (if needed)
docker cp meepleai-redis:/data/appendonly.aof "redis_aof_$(date +%Y%m%d).aof"
```

Restore

```
# Stop Redis
docker compose stop redis

# Copy backup into container volume
docker cp redis_backup_20260316.rdb meepleai-redis:/data/dump.rdb

# Start Redis
docker compose start redis

# Verify
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" DBSIZE
```

Backup policy: Redis is used as a cache. Full restore from backup is rarely needed — the API repopulates the cache on startup. Back up weekly for convenience.

Scaling & Performance Tuning

Scenario	maxmemory	Notes
Development	256 MB	Minimal cache
Small prod (~100 users)	512 MB	Default sufficient
Medium prod (~1K users)	768 MB	Current config
Large prod (~10K users)	1.5 GB	Increase container limit to 2G

```
# Check current maxmemory at runtime
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" CONFIG GET maxmemory

# Change maxmemory at runtime (does not survive restart)
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" CONFIG SET maxmemory 1073741824
```

```
# Persist change: edit infra/redis/redis.conf and restart
```

Upgrade & Migration

```
# 1. Backup
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" BGSAVE
docker cp meepleai-redis:/data/dump.rdb redis_pre_upgrade.rdb

# 2. Update image in docker-compose.yml
# image: redis:7.6-alpine

# 3. Pull and recreate
docker compose pull redis
docker compose up -d redis

# 4. Verify
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" INFO server | grep redis_version
```

Troubleshooting

Symptom	Diagnosis	Fix
PONG not returned	Wrong password	Verify `redis.secret`
High eviction rate	`INFO stats` → `evicted_keys`	Increase `maxmemory` or review key TTLs
OOM kill	Container exceeds memory limit	Increase container limit + `maxmemory`
Connection refused	Redis not started	`docker compose restart redis`
Slow commands	`SLOWLOG GET 10`	Optimize key patterns, avoid `KEYS *`
AOF rewrite stuck	Disk I/O or memory pressure	Check disk space, restart Redis

```
# Flush all keys (DANGER — clears entire cache)
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" FLUSHALL

# View slow log
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" SLOWLOG GET 10
```

Disaster Recovery

- **RTO**: < 5 minutes (restart, cache warms from API)

- **RPO**: N/A (cache is ephemeral; data lives in PostgreSQL)
- **Recovery**: Simply restart Redis; API repopulates cache automatically

Cost Notes

- Redis is open source, zero license cost
- RAM only: ~768 MB allocation
- No persistent storage cost (AOF is small for cache workloads)

---

7. API Service (.NET 9)

Overview

Property	Value
Image	Custom build from `apps/api/src/Api/Dockerfile`
Container	`meepleai-api`
Port	8080
Volumes	`pdf_uploads` → `/app/pdf_uploads`, `infra/secrets` → `/app/infra/secrets:ro`
CPU / RAM Limit	2.0 / 4G
CPU / RAM Reserved	1.0 / 2G
Health Check	`curl --fail http://localhost:8080/` (10s interval, 5s timeout, 12 retries, 20s start period)
Depends On	postgres (healthy), qdrant (started), redis (healthy)
Entrypoint	`/scripts/load-secrets-env.sh` → `dotnet Api.dll`

Daily Operations

```
# Check health
curl http://localhost:8080/

# API documentation (browser)
# http://localhost:8080/scalar/v1

# View recent logs
docker logs meepleai-api --tail=100

# Follow logs in real-time
```

```
docker logs meepleai-api -f

# Check resource usage
docker stats meepleai-api --no-stream

# Restart without downtime (depends on orchestrator)
docker compose restart api

# Run database migrations (from host)
cd apps/api/src/Api && dotnet ef database update
```

Backup & Restore

The API is stateless except for:

1. **pdf_uploads volume**: Uploaded PDF files stored on disk
2. **Database state**: Managed by PostgreSQL (see Section 4)

```
# Backup pdf_uploads volume
docker run --rm -v meepleai_pdf_uploads:/data -v $(pwd):/backup alpine \
  tar czf /backup/pdf_uploads_$(date +%Y%m%d).tar.gz -C /data .

# Restore pdf_uploads volume
docker run --rm -v meepleai_pdf_uploads:/data -v $(pwd):/backup alpine \
  sh -c "cd /data && tar xzf /backup/pdf_uploads_20260316.tar.gz"
```

Scaling & Performance Tuning

```
# Monitor request latency and error rates via Prometheus
# PromQL: histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))
# PromQL: rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m]) * 10

# Check GC pressure (.NET specific)
docker exec meepleai-api dotnet-counters monitor --process-id 1 System.Runtime

# Increase worker threads if needed (via environment variable)
# DOTNET_ThreadPool_MinThreads=50
```

Load	CPU Limit	RAM Limit	Notes
Dev / < 100 users	1.0	2G	Default sufficient
Small prod (~500)	2.0	4G	Current config
Medium prod (~2K)	4.0	8G	Consider horizontal scaling

Upgrade & Migration

```
# 1. Build new image
cd apps/api && docker build -f src/Api/Dockerfile -t meepleai-api:new .

# 2. Apply database migrations first
cd src/Api && dotnet ef database update

# 3. Recreate container with new image
cd infra && docker compose up -d --build api

# 4. Verify health
curl http://localhost:8080/
docker logs meepleai-api --tail=20
```

Troubleshooting

Symptom	Diagnosis	Fix
503 during startup	Dependencies not ready	Check postgres, qdrant, redis health
High memory (> 3G)	Memory leak or GC pressure	Check `dotnet-counters`, restart
500 errors	`docker logs meepleai-api`	Check stack traces, DB connectivity
Slow responses	P95 > 1s	Check DB queries, Redis hit rate
Can't reach DB	Connection string wrong	Verify `database.secret`
Secret load fails	Missing `.secret` files	Run `setup-secrets.ps1 -SaveGenerated`
PDF upload fails	Volume permission issue	Check `pdf_uploads` mount

```
# Force restart
docker compose restart api

# Rebuild from scratch
docker compose up -d --build --force-recreate api

# Check EF Core migration status
cd apps/api/src/Api && dotnet ef migrations list
```

Disaster Recovery

- **RTO**: 5-10 minutes (rebuild container, apply migrations)
- **RPO**: N/A (stateless; state is in PostgreSQL + pdf_uploads volume)

- **Recovery**: Rebuild image → start container → verify health

Cost Notes

- .NET 9 runtime is free and open source
- Primary cost is CPU/RAM allocation (2.0/4G)
- Build time: ~2-3 minutes for Docker image

---

8. Web Service (Next.js 16)

Overview

Property	Value
Image	Custom build from `apps/web/Dockerfile`
Container	`meepleai-web`
Port	3000
Volume	None
CPU / RAM Limit	1.0 / 1G
CPU / RAM Reserved	0.5 / 512M
Health Check	`curl -f http://localhost:3000/` (15s interval, 5s timeout, 12 retries, 60s start period)
Depends On	api (healthy)

Daily Operations

```
# Check health
curl http://localhost:3000/

# View logs
docker logs meepleai-web --tail=50

# Check resource usage
docker stats meepleai-web --no-stream

# Restart
docker compose restart web
```

Backup & Restore

The web service is fully stateless — no backup needed. All state is in the API/database.

Scaling & Performance Tuning

Load	CPU	RAM	Notes
Dev	0.5	512M	SSR minimal load
Small prod	1.0	1G	Current config
Medium prod	2.0	2G	Higher SSR concurrency

Next.js performance considerations:

- Enable ISR (Incremental Static Regeneration) for heavy pages
- Monitor SSR memory via `docker stats`
- Use `NEXT_PUBLIC_*` environment variables for client-side config

Upgrade & Migration

```
# 1. Install dependencies and build
cd apps/web && pnpm install && pnpm build

# 2. Rebuild Docker image and restart
cd infra && docker compose up -d --build web

# 3. Verify
curl http://localhost:3000/
```

Troubleshooting

Symptom	Diagnosis	Fix
White screen	JS bundle error	Check browser console, rebuild: `rm -rf .next && pnpm build`
503 on load	API not healthy	Check API health first
Slow first load	SSR compilation	Pre-warm after deploy
OOM kill	SSR memory spike	Increase RAM limit to 2G
Hydration error	Server/client mismatch	Check for `Math.random()` or `Date.now()` in components

Build fails	Dependency issue	`rm -rf node_modules .next && pnpm install && pnpm build`

Disaster Recovery

- **RTO**: 2-5 minutes (rebuild and restart)
- **RPO**: N/A (stateless)

Cost Notes

- Zero license cost (Next.js is open source)
- Lightweight resource usage (1.0 CPU / 1G RAM)
- Build time: ~1-2 minutes

---

9. Ollama LLM Runtime

Overview

Property	Value
Image	`ollama/ollama:0.5.4`
Container	`meepleai-ollama`
Port	11434
Profile	`ai`
Volume	`ollama_data` → `/root/.ollama`
CPU / RAM Limit	4.0 / 8G
CPU / RAM Reserved	2.0 / 4G
Health Check	`ollama list` (10s interval, 5s timeout, 5 retries, 20s start period)

Init container: `ollama-pull` (curlimages/curl:8.12.1) runs once after Ollama is healthy to pull the `nomic-embed-text` model. CPU 1.0 / RAM 512M, `restart: "no"`.

Daily Operations

```
# Check running models
docker exec meepleai-ollama ollama list

# Test model inference
docker exec meepleai-ollama ollama run nomic-embed-text "test"
```

```
# Check model storage
docker exec meepleai-ollama du -sh /root/.ollama/models/

# Pull a new model
docker exec meepleai-ollama ollama pull nomic-embed-text

# View logs
docker logs meepleai-ollama --tail=50

# Check GPU availability (if applicable)
docker exec meepleai-ollama ollama ps
```

Backup & Restore

```
# Backup model data (large — several GB)
docker run --rm -v meepleai_ollama_data:/data -v $(pwd):/backup alpine \
  tar czf /backup/ollama_models_$(date +%Y%m%d).tar.gz -C /data .

# Restore
docker run --rm -v meepleai_ollama_data:/data -v $(pwd):/backup alpine \
  sh -c "cd /data && tar xzf /backup/ollama_models_20260316.tar.gz"
```

Alternative: Skip backup entirely — models can be re-pulled. The `ollama-pull` init container handles this automatically.

Scaling & Performance Tuning

Configuration	CPU	RAM	Use Case
CPU-only (default)	4.0	8G	Development, small models
GPU passthrough	4.0	8G + GPU	Production inference

To enable GPU passthrough (NVIDIA):

```
# In compose override
ollama:
  deploy:
    resources:
      reservations:
        devices:
          - driver: nvidia
```

```
            count: 1
            capabilities: [gpu]
```

Upgrade & Migration

```
# 1. Note current models
docker exec meepleai-ollama ollama list

# 2. Update image tag
# image: ollama/ollama:0.6.0

# 3. Pull and recreate (models persist in volume)
docker compose pull ollama
docker compose --profile ai up -d ollama

# 4. Verify models still available
docker exec meepleai-ollama ollama list
```

Troubleshooting

Symptom	Diagnosis	Fix
Model not found	`ollama list` empty	`ollama pull nomic-embed-text`
OOM kill	Model too large for RAM	Use smaller model or increase RAM
Slow inference	CPU-only, large model	Enable GPU or use quantized models
Pull timeout	Network issue	Retry: `docker exec meepleai-ollama ollama pull nomic-embed-text`
ollama-pull fails	Ollama not healthy yet	Check Ollama health, increase start_period

Disaster Recovery

- **RTO**: 5-15 minutes (restart + model pull)
- **RPO**: N/A (models are re-downloadable)

Cost Notes

- Ollama is free and open source
- RAM-heavy: 8G reserved for model inference

- Model storage: 1-5 GB per model in volume
- GPU recommended for production inference workloads

---

10. Embedding Service

Overview

Property	Value
Image	Custom build from `apps/embedding-service/Dockerfile`
Container	`meepleai-embedding`
Port	8000
Profile	`ai`
Volume	None (model cached in image or downloaded at startup)
CPU / RAM Limit	2.0 / 4G
CPU / RAM Reserved	1.0 / 2G
Health Check	`curl -f http://localhost:8000/health` (30s interval, 10s timeout, 3 retries, 60s start period)

The embedding service runs `sentence-transformers` models to generate 1024-dimensional vectors from text. It is a dependency of the orchestration service.

Daily Operations

```
# Health check
curl http://localhost:8000/health

# Test embedding generation
curl -X POST http://localhost:8000/embed \
  -H "Content-Type: application/json" \
  -d '{"text": "test sentence"}'

# View logs
docker logs meepleai-embedding --tail=50

# Resource usage
docker stats meepleai-embedding --no-stream
```

Backup & Restore

No persistent state. Models are baked into the Docker image or downloaded at startup. Rebuild the image to restore.

Scaling & Performance Tuning

Load	CPU	RAM	Notes
Dev	1.0	2G	Single model loaded
Prod (< 100 req/min)	2.0	4G	Current config
Prod (> 100 req/min)	4.0	8G	Consider batching

Performance tips:

- Batch embedding requests where possible
- Model loads into RAM once at startup (~1-2 GB)
- Inference is CPU-bound; GPU dramatically improves throughput

Upgrade & Migration

```
# Rebuild with updated model or code
cd infra && docker compose --profile ai up -d --build embedding-service

# Verify
curl http://localhost:8000/health
```

Troubleshooting

Symptom	Diagnosis	Fix
Slow startup (>60s)	Model downloading	Pre-bake model in Dockerfile
OOM kill	Model + batches exceed RAM	Reduce batch size or increase RAM
500 on /embed	Model load failure	Check logs, rebuild image
Connection refused	Service not started	`docker compose --profile ai up -d embedding-service`

Disaster Recovery

- **RTO**: 2-5 minutes (rebuild + restart)

- **RPO**: N/A (stateless)

Cost Notes

- sentence-transformers is open source
- RAM: ~2 GB for model, ~2 GB for inference headroom
- CPU-bound inference; GPU optional but beneficial

---

11. Document Processing Services

This section covers two document processing services that extract text and structure from PDFs.

11a. Unstructured Service

Overview

Property	Value
Image	Custom build from `apps/unstructured-service/Dockerfile`
Container	`meepleai-unstructured`
Port	8001
Profile	`ai`
Volume	`unstructured_temp` → `/tmp/pdf-extraction`
CPU / RAM Limit	2.0 / 2G
CPU / RAM Reserved	1.0 / 1G
Health Check	`curl -f http://localhost:8001/health` (30s interval, 10s timeout, 3 retries, 20s start period)

The Unstructured service handles PDF parsing, text extraction, and document chunking using the Unstructured Python library.

Daily Operations

```
# Health check
curl http://localhost:8001/health

# View logs
docker logs meepleai-unstructured --tail=50
```

```
# Check temp directory size
docker exec meepleai-unstructured du -sh /tmp/pdf-extraction/

# Clean temp files (safe — these are intermediate processing files)
docker exec meepleai-unstructured rm -rf /tmp/pdf-extraction/*
```

Backup & Restore

No persistent state. The `unstructured_temp` volume holds temporary processing files that can be safely deleted. Source PDFs are in the API's `pdf_uploads` volume.

Scaling & Performance Tuning

Load	CPU	RAM	Notes
Dev	1.0	1G	Light PDF processing
Prod (< 50 PDFs/day)	2.0	2G	Current config
Prod (> 50 PDFs/day)	4.0	4G	PDF processing is CPU-heavy

Upgrade & Migration

```
cd infra && docker compose --profile ai up -d --build unstructured-service
curl http://localhost:8001/health
```

Troubleshooting

Symptom	Diagnosis	Fix
PDF extraction fails	Check logs for parsing errors	`docker logs meepleai-unstructured`
Temp disk full	Large PDFs accumulate	Clean temp: `rm -rf /tmp/pdf-extraction/*`
OOM on large PDF	PDF exceeds RAM limit	Process smaller chunks or increase RAM
Slow extraction	CPU-bound OCR	Increase CPU limit

Disaster Recovery

- **RTO**: 2-5 minutes (rebuild)
- **RPO**: N/A (stateless; source PDFs in API volume)

Cost Notes

- Open source (Unstructured library)

- CPU-intensive during PDF processing; idle otherwise
- Temp storage: negligible (cleaned after processing)

11b. SmolDocling Service

Overview

Property	Value
Image	Custom build from `apps/smoldocling-service/Dockerfile`
Container	`meepleai-smoldocling`
Port	8002
Profile	`ai`
Volumes	`smoldocling_temp` → `/tmp/pdf-processing`, `smoldocling_models` → `/root/.cache/huggingface`
CPU / RAM Limit	2.0 / 4G
CPU / RAM Reserved	1.0 / 2G
Health Check	`curl -f http://localhost:8002/health` (60s interval, 15s timeout, 3 retries, 120s start period)

SmolDocling provides advanced PDF processing with OCR capabilities using Hugging Face models. It has a longer startup time (up to 120s) due to model loading.

Daily Operations

```
# Health check
curl http://localhost:8002/health

# View logs
docker logs meepleai-smoldocling --tail=50

# Check model cache size
docker exec meepleai-smoldocling du -sh /root/.cache/huggingface/

# Check temp directory
docker exec meepleai-smoldocling du -sh /tmp/pdf-processing/
```

Backup & Restore

```
# Backup model cache (avoids re-download on rebuild)
docker run --rm -v meepleai_smoldocling_models:/data -v $(pwd):/backup alpine \
  tar czf /backup/smoldocling_models_$(date +%Y%m%d).tar.gz -C /data .
```

Models can be re-downloaded from Hugging Face if the cache is lost — backup is optional.

Scaling & Performance Tuning

Load	CPU	RAM	Notes
Dev	1.0	2G	Slow but functional
Prod	2.0	4G	Current config
Heavy OCR	4.0	8G	GPU recommended

Upgrade & Migration

```
cd infra && docker compose --profile ai up -d --build smoldocling-service
# Allow 120s for model loading
sleep 30 && curl http://localhost:8002/health
```

Troubleshooting

Symptom	Diagnosis	Fix
Very slow startup	Model downloading from HF	Check `smoldocling_models` volume exists
OOM kill	Model + inference exceeds 4G	Increase RAM limit to 6G
Health check timeout	120s start_period too short	Increase start_period to 180s
Model download fails	Network/HF outage	Retry; check HF status page

Disaster Recovery

- **RTO**: 5-15 minutes (rebuild + model download)
- **RPO**: N/A (stateless)

Cost Notes

- Open source (SmolDocling / Hugging Face)
- Model cache: 1-3 GB in `smoldocling_models` volume
- RAM-heavy: 4G minimum for model + inference

---

12. Reranker Service

Overview

Property	Value
Image	Custom build from `apps/reranker-service/Dockerfile`
Container	`meepleai-reranker`
Port	8003
Profile	`ai`
Volume	`reranker_models` → `/home/reranker/.cache/huggingface`
CPU / RAM Limit	2.0 / 2G
CPU / RAM Reserved	1.0 / 1G
Health Check	`curl -f http://localhost:8003/health` (30s interval, 10s timeout, 3 retries, 120s start period)

The reranker service uses a cross-encoder model to re-rank search results for improved RAG quality. It is a dependency of the orchestration service.

Daily Operations

```
# Health check
curl http://localhost:8003/health

# View logs
docker logs meepleai-reranker --tail=50

# Check model cache
docker exec meepleai-reranker du -sh /home/reranker/.cache/huggingface/

# Resource usage
docker stats meepleai-reranker --no-stream
```

Backup & Restore

```
# Backup model cache
docker run --rm -v meepleai_reranker_models:/data -v $(pwd):/backup alpine \
```

```
tar czf /backup/reranker_models_$(date +%Y%m%d).tar.gz -C /data .
```

Models re-downloadable from Hugging Face — backup is convenience only.

Scaling & Performance Tuning

Load	CPU	RAM	Notes
Dev	1.0	1G	Single-query reranking
Prod (< 50 req/min)	2.0	2G	Current config
Prod (> 50 req/min)	4.0	4G	Batch reranking, increase concurrency

Cross-encoder models are more compute-intensive than bi-encoders. Reranking is typically the bottleneck in the RAG pipeline.

Upgrade & Migration

```
cd infra && docker compose --profile ai up -d --build reranker-service
sleep 30 && curl http://localhost:8003/health
```

Troubleshooting

Symptom	Diagnosis	Fix
Slow startup	Model download	Ensure `reranker_models` volume persists
OOM kill	Model exceeds 2G	Increase RAM to 4G
Low reranking quality	Model version issue	Rebuild with updated model
Connection timeout	Start period too short	Increase from 120s to 180s

Disaster Recovery

- **RTO**: 5-15 minutes (rebuild + model download)
- **RPO**: N/A (stateless)

Cost Notes

- Open source (cross-encoder from Hugging Face)
- Model cache: 500 MB - 1 GB
- Inference is CPU-bound; GPU dramatically speeds up reranking

---

13. Orchestration Service

Overview

Property	Value
Image	Custom build from `apps/orchestration-service/Dockerfile`
Container	`meepleai-orchestrator`
Port	8004
Profile	`ai`
Volume	None
CPU / RAM Limit	2.0 / 4G
CPU / RAM Reserved	1.0 / 2G
Health Check	`curl -f http://localhost:8004/health` (30s interval, 10s timeout, 3 retries, 60s start period)
Depends On	embedding-service (started), reranker-service (started), postgres (healthy), redis (healthy)

The orchestration service coordinates the AI pipeline: it receives queries from the API, calls the embedding service for vector search, calls the reranker for result quality, and manages multi-agent AI workflows.

Daily Operations

```
# Health check
curl http://localhost:8004/health

# View logs
docker logs meepleai-orchestrator --tail=50

# Resource usage
docker stats meepleai-orchestrator --no-stream
```

Backup & Restore

No persistent state. The orchestration service is a pure coordinator.

Scaling & Performance Tuning

Load	CPU	RAM	Notes
Dev	1.0	2G	Low concurrency
Prod (< 100 req/min)	2.0	4G	Current config
Prod (> 100 req/min)	4.0	8G	Increase for concurrent pipelines

The orchestrator's performance is bounded by its downstream services (embedding, reranker, Qdrant). Optimize those first.

Upgrade & Migration

```
cd infra && docker compose --profile ai up -d --build orchestration-service
curl http://localhost:8004/health
```

Troubleshooting

Symptom	Diagnosis	Fix
503 at startup	Dependencies not ready	Check embedding + reranker + postgres + redis health
Timeout on queries	Downstream service slow	Check embedding/reranker latency
Pipeline errors	`docker logs meepleai-orchestrator`	Check full pipeline: embed → search → rerank
Memory growth	Accumulating context	Restart: `docker compose restart orchestration-service`

Disaster Recovery

- **RTO**: 2-5 minutes (rebuild + restart)
- **RPO**: N/A (stateless coordinator)

Cost Notes

- Custom Python service, no license cost
- RAM: 4G allocation covers concurrent pipeline management
- Depends on all other AI services being available

---

14. Monitoring Stack

Overview

The monitoring stack runs under the `monitoring` profile and consists of five services.

Service	Role	Port	CPU/RAM
Prometheus	Metrics collection & alerting	9090	1.0 / 2G
Grafana	Dashboards & visualization	3001	1.0 / 1G
Alertmanager	Alert routing & notification	9093	0.5 / 512M
Node Exporter	Host-level metrics	9100	0.5 / 256M
cAdvisor	Container-level metrics	8080	0.5 / 512M

Starting the Monitoring Stack

```
cd infra
docker compose -f docker-compose.yml -f compose.dev.yml --profile monitoring up -d
```

Prometheus

Configuration

Setting	Value
Image	`prom/prometheus:v3.7.0`
Container	`meepleai-prometheus`
Config	`infra/prometheus.yml` (mounted read-only)
Rules	`infra/prometheus-rules.yml` (mounted read-only)
Volume	`prometheus_data` → `/prometheus`
Retention	30 days
Max Storage	5 GB
Scrape Interval	15 seconds
Health Check	`wget --spider -q http://localhost:9090/-/healthy`

Alert Rule Categories

Category	Examples

api-performance	Latency P95 > 1s, throughput drops
cache-performance	Redis hit rate < 70%
database-health	Connections > 80%, replication lag
infrastructure	CPU > 80%, Memory > 85%, Disk < 20%
http-retry	Retry rate exceeding threshold
pdf-processing	Processing failures, queue backlog
prompt-management	Template errors, token overflows
quality-metrics	RAG quality below threshold
vector-search	Qdrant latency, collection errors
e2e-tests	Test suite failures

Key Alert Thresholds

Metric	Threshold	Duration	Severity
CPU usage	> 80%	10 min	Warning
Memory usage	> 85%	10 min	Warning
Disk free	< 20%	30 min	Critical
Error rate	> 5%	5 min	Critical
Latency P95	> 1s	10 min	Warning
Cache hit rate	< 70%	15 min	Warning

Key PromQL Queries

```
# Error rate (5xx / total)
rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m]) * 100

# Latency P95
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

# Latency P99
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))

# Cache hit rate
rate(redis_keyspace_hits_total[5m]) / (rate(redis_keyspace_hits_total[5m]) + rate(redis_key

# Request throughput
sum(rate(http_requests_total[5m]))
```

```
# CPU usage by container
rate(container_cpu_usage_seconds_total[5m]) * 100

# Memory usage by container
container_memory_usage_bytes / container_spec_memory_limit_bytes * 100

# PostgreSQL active connections
pg_stat_activity_count{state="active"}

# Qdrant collection size
qdrant_collection_vectors_count{collection="meepleai_documents"}
```

Daily Operations

```
# Check Prometheus health
curl http://localhost:9090/-/healthy

# Reload config without restart
curl -X POST http://localhost:9090/-/reload

# Check active alerts
curl http://localhost:9090/api/v1/alerts

# Check targets (scrape status)
curl http://localhost:9090/api/v1/targets

# View logs
docker logs meepleai-prometheus --tail=50

# Check storage usage
docker exec meepleai-prometheus df -h /prometheus
```

Grafana

Configuration

Setting	Value
Image	`grafana/grafana:11.4.0`
Container	`meepleai-grafana`
Port	3001 (mapped from container 3000)
Volume	`grafana_data` → `/var/lib/grafana`

Provisioning	`infra/grafana-datasources.yml`, `infra/grafana-dashboards.yml`
Dashboards	`infra/dashboards/` (mounted read-only)
Health Check	`wget --spider -q http://localhost:3000/api/health`

Daily Operations

```
# Access Grafana UI
# http://localhost:3001 (default admin/admin on first login)

# Check health
curl http://localhost:3001/api/health

# API: list dashboards
curl -H "Authorization: Bearer $GRAFANA_API_KEY" http://localhost:3001/api/search

# View logs
docker logs meepleai-grafana --tail=50
```

Backup Grafana Data

```
# Backup dashboards and settings
docker run --rm -v meepleai_grafana_data:/data -v $(pwd):/backup alpine \
  tar czf /backup/grafana_data_$(date +%Y%m%d).tar.gz -C /data .
```

Alertmanager

Setting	Value
Image	`prom/alertmanager:v0.27.0`
Container	`meepleai-alertmanager`
Port	9093
Config	`infra/alertmanager.yml`
Volume	`alertmanager_data`
Health Check	`wget --spider -q http://localhost:9093/-/healthy`

```
# Check health
curl http://localhost:9093/-/healthy
```

```
# View active alerts
curl http://localhost:9093/api/v2/alerts

# Silence an alert (example)
curl -X POST http://localhost:9093/api/v2/silences -H "Content-Type: application/json" \
  -d '{
    "matchers": [{"name": "alertname", "value": "HighCPU", "isRegex": false}],
    "startsAt": "2026-03-16T00:00:00Z",
    "endsAt": "2026-03-17T00:00:00Z",
    "createdBy": "operator",
    "comment": "Planned maintenance"
  }'

# View logs
docker logs meepleai-alertmanager --tail=50
```

Node Exporter

Setting	Value
Image	`prom/node-exporter:v1.8.2`
Container	`meepleai-node-exporter`
Port	9100
Volumes	`/` → `/host:ro`
CPU / RAM	0.5 / 256M

Exposes host-level metrics: CPU, memory, disk, network. Scraped by Prometheus.

```
# Check metrics endpoint
curl http://localhost:9100/metrics | head -20
```

cAdvisor

Setting	Value
Image	`gcr.io/cadvisor/cadvisor:v0.49.1`
Container	`meepleai-cadvisor`
Port	8080
Volumes	Docker socket, /sys, /var/lib/docker (all read-only)

CPU / RAM	0.5 / 512M
Security	`no-new-privileges=true`

Exposes per-container CPU, memory, network, and I/O metrics. Scraped by Prometheus.

```
# Check cAdvisor UI
# http://localhost:8080/containers/

# Check metrics
curl http://localhost:8080/metrics | head -20
```

Troubleshooting

Symptom	Diagnosis	Fix
No metrics in Grafana	Prometheus target down	Check `http://localhost:9090/targets`
Alert not firing	Rule syntax error	`curl -X POST http://localhost:9090/-/reload`
Grafana 502	Prometheus unreachable	Check Prometheus health
Disk full (Prometheus)	Exceeded 5GB retention	Reduce retention or clean old data
cAdvisor crash	Docker socket access	Verify Docker socket mount

Disaster Recovery

- **RTO**: 5-10 minutes (restart stack)
- **RPO**: 30 days of metrics in Prometheus volume; Grafana dashboards auto-provisioned from files

Cost Notes

- All components open source
- Total monitoring RAM: ~4.3 GB
- Prometheus storage: up to 5 GB (30-day retention)
- Grafana storage: < 100 MB

---

15. Automation Services

Overview

The automation stack runs under the `automation` profile.

Service	Role	Port(s)	CPU/RAM
n8n	Workflow automation	5678	1.0 / 1G
Mailpit	Email testing (SMTP trap)	8025 (UI), 1025 (SMTP)	0.5 / 128M

n8n Workflow Engine

Configuration

Property	Value
Image	`n8nio/n8n:1.114.4`
Container	`meepleai-n8n`
Port	5678
Profile	`automation`
Depends On	postgres (healthy)
CPU / RAM Limit	1.0 / 1G
Entrypoint	`/scripts/load-secrets-env.sh` → `n8n`

Daily Operations

```
# Access n8n UI
# http://localhost:5678

# View logs
docker logs meepleai-n8n --tail=50

# Restart
docker compose --profile automation restart n8n
```

Backup & Restore

n8n stores workflows in PostgreSQL (shared database). Workflow data is backed up with the PostgreSQL dump (Section 4).

```
# Export specific workflows via API
curl http://localhost:5678/api/v1/workflows \
  -H "X-N8N-API-KEY: $N8N_API_KEY" > n8n_workflows_backup.json
```

Troubleshooting

Symptom	Diagnosis	Fix
Can't access UI	Port not exposed	Check compose override
Workflow fails	Check execution logs in n8n UI	Fix workflow nodes
DB connection error	PostgreSQL not healthy	Check postgres health
Authentication issues	Secret not loaded	Verify secrets mount

Mailpit (Email Testing)

Configuration

Property	Value
Image	`axllent/mailpit:v1.22`
Container	`mailpit`
Ports	8025 (Web UI), 1025 (SMTP)
Profile	`automation`
Volume	`mailpit_data` → `/data`
CPU / RAM Limit	0.5 / 128M
Health Check	`wget` on API + `nc` on SMTP (10s interval)

Mailpit is a local SMTP server that catches all outgoing email. Configure the API to use `mailpit:1025` as the SMTP server in development.

Daily Operations

```
# Access Mailpit UI (view caught emails)
# http://localhost:8025

# Check SMTP is accepting connections
nc -z localhost 1025

# View logs
docker logs mailpit --tail=20

# API: list messages
curl http://localhost:8025/api/v1/messages
```

Troubleshooting

Symptom	Diagnosis	Fix
No emails appearing	API not configured to use mailpit SMTP	Set SMTP host to `mailpit:1025`
SMTP connection refused	Mailpit not running	`docker compose --profile automation up -d mailpit`
UI not loading	Port 8025 not exposed	Check compose override

Disaster Recovery

- **RTO**: < 5 minutes
- **RPO**: n8n workflows backed up with PostgreSQL; Mailpit data is ephemeral

Cost Notes

- Both services open source
- Very lightweight: 1.5 CPU / 1.1 GB RAM total
- Mailpit is development-only; not used in production

---

16. Storage Services

Overview

The storage stack runs under the `storage` profile.

Service	Role	Port(s)	CPU/RAM
MinIO	S3-compatible object storage	9000 (API), 9001 (Console)	0.5 / 512M
minio-init	Bucket initialization (runs once)	-	-

MinIO S3 Storage

Configuration

Property	Value
Image	`minio/minio:RELEASE.2024-11-07T00-52-20Z`
Container	`meepleai-minio`
Ports	9000 (S3 API), 9001 (Web Console)

Profile	`storage`
Volume	`minio_data` → `/data`
Command	`server /data --console-address ":9001"`
CPU / RAM Limit	0.5 / 512M
Health Check	`mc ready local`

Init container (`minio-init`): Uses `minio/mc:RELEASE.2024-11-17T19-35-25Z` to create the `meepleai-uploads` bucket on first startup. Runs once and exits.

Daily Operations

```
# Access MinIO Console
# http://localhost:9001 (login with MINIO_ROOT_USER/MINIO_ROOT_PASSWORD)

# Check health
curl http://localhost:9000/minio/health/ready

# List buckets (using mc CLI)
docker exec meepleai-minio mc alias set local http://localhost:9000 minioadmin minioadmin12
docker exec meepleai-minio mc ls local/

# Check bucket size
docker exec meepleai-minio mc du local/meepleai-uploads

# View logs
docker logs meepleai-minio --tail=50
```

Backup & Restore

```
# Backup MinIO data
docker run --rm -v meepleai_minio_data:/data -v $(pwd):/backup alpine \
  tar czf /backup/minio_data_$(date +%Y%m%d).tar.gz -C /data .

# Restore
docker compose stop minio
docker run --rm -v meepleai_minio_data:/data -v $(pwd):/backup alpine \
  sh -c "cd /data && tar xzf /backup/minio_data_20260316.tar.gz"
docker compose start minio
```

S3 Integration with API

The API uses `IBlobStorageService` with a factory pattern. Configure in `infra/secrets/storage.secret`:

```
STORAGE_PROVIDER=s3
S3_ENDPOINT=http://minio:9000        # Internal Docker network
S3_ACCESS_KEY=minioadmin
S3_SECRET_KEY=minioadmin123
S3_BUCKET_NAME=meepleai-uploads
S3_REGION=auto
S3_PRESIGNED_URL_EXPIRY=3600
S3_FORCE_PATH_STYLE=true             # Required for MinIO
```

For production with Cloudflare R2:

```
STORAGE_PROVIDER=s3
S3_ENDPOINT=https://xxx.r2.cloudflarestorage.com
S3_ACCESS_KEY=your_r2_access_key
S3_SECRET_KEY=your_r2_secret_key
S3_BUCKET_NAME=meepleai-uploads
S3_REGION=auto
S3_PRESIGNED_URL_EXPIRY=3600
S3_FORCE_PATH_STYLE=false
```

Troubleshooting

Symptom	Diagnosis	Fix
Bucket not found	minio-init didn't run	`docker compose --profile storage up -d minio-init`
Access denied	Wrong credentials	Check `storage.secret`
Upload fails	S3_FORCE_PATH_STYLE wrong	Set to `true` for MinIO
Console not loading	Port 9001 not exposed	Check compose override

Disaster Recovery

- **RTO**: 5-10 minutes
- **RPO**: Depends on backup frequency; files also in `pdf_uploads` volume as fallback

Cost Notes

- MinIO is open source

- For production, consider Cloudflare R2 ($0.015/GB/month, free egress)
- Local MinIO uses 512M RAM + storage for objects

---

17. Incident Response

Severity Levels

Level	Name	Definition	Response Time	Examples
P1	Critical	Service down or > 5% error rate	< 15 min	API unreachable, database corruption, all users affected
P2	High	Core feature degraded	< 1 hour	Search broken, PDF upload failing, slow API
P3	Medium	Secondary feature broken	< 4 hours	Monitoring gap, n8n workflow failing, email not working
P4	Low	Minor issue, no user impact	< 24 hours	Log noise, cosmetic UI bug, stale metrics

P1 Critical Incident Procedure

```
1. DETECT
   ├── Alertmanager notification
   ├── Health check failure
   └── User report

2. ASSESS (< 5 min)
   ├── docker compose ps                    # Which services are down?
   ├── docker stats --no-stream             # Resource exhaustion?
   ├── docker logs <container> --tail=50    # Error messages?
   └── Determine blast radius

3. CONTAIN (< 10 min)
   ├── If API down:     docker compose restart api
   ├── If DB down:      docker compose restart postgres
   ├── If Redis down:   docker compose restart redis
   ├── If OOM:          docker compose restart <service>
   └── If disk full:    docker system prune -f

4. RESOLVE
   ├── Apply fix (config change, restart, rollback)
   ├── Verify health:  docker compose ps
```

```
   ├── Check logs:     docker logs <container> --tail=20
   └── Run smoke test: curl http://localhost:8080/

5. POST-MORTEM
   ├── Document: what happened, why, how fixed
   ├── Identify: prevention measures
   └── Implement: monitoring improvements
```

Quick Response Commands

```
# == DIAGNOSTIC ==

# What's running and what's not?
docker compose ps

# Which containers are using the most resources?
docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"

# Check all health statuses
docker inspect --format='{{.Name}}: {{.State.Health.Status}}' $(docker ps -q) 2>/dev/null

# Check disk space
df -h /var/lib/docker
docker system df

# == RECOVERY ==

# Restart everything (graceful)
docker compose down && docker compose up -d

# Restart specific service
docker compose restart <service-name>

# Force recreate (clears container state)
docker compose up -d --force-recreate <service-name>

# Emergency: kill and restart
docker kill <container-name> && docker compose up -d <service-name>

# == INVESTIGATION ==

# Check recent logs for errors
docker compose logs --tail=100 --since=5m 2>&1 | grep -i error

# Check container exit codes
docker ps -a --format "table {{.Names}}\t{{.Status}}" | grep -v "Up "
```

```
# Check OOM kills
dmesg | grep -i "oom\|killed" | tail -20
```

Escalation Matrix

Situation	Action
Simple restart fixes it	Monitor for recurrence
Same issue 3x in 24h	Root cause analysis required
Data loss suspected	Stop writes, investigate, restore from backup
Security breach suspected	Isolate affected service, rotate all secrets
Infrastructure failure	Consider VPS migration (see DR section)

Communication Template

```
INCIDENT: [P1/P2/P3/P4] [Service Name] - [Brief Description]
TIME: [Detection time]
STATUS: [Investigating / Identified / Resolved]
IMPACT: [What users are experiencing]
ACTIONS: [What is being done]
ETA: [When resolution expected]
```

---

18. Maintenance & Disaster Recovery

Scheduled Maintenance

Daily Checklist

Task	Command	Automated?
Health checks	`docker compose ps`	Yes (health checks)
Log review	`docker compose logs --tail=50 --since=24h`	No
Backup verification	Check backup file exists and size > 0	Yes (cron)
Alert review	Check Alertmanager / Grafana	No

Weekly Checklist

Task	Command	Notes
Disk usage	`docker system df` + `df -h`	Alert if > 80%
Backup integrity	Restore test to temp DB	Verify row counts
SSL certificate expiry	`openssl s_client` or Traefik logs	Must be > 14 days
Volume growth	Compare `docker system df` week-over-week	Trend analysis

Monthly Checklist

Task	Command	Notes
Docker image updates	`docker compose pull`	Review changelogs first
OS patches	`apt update && apt upgrade`	Schedule maintenance window
VACUUM FULL	`docker exec meepleai-postgres psql -U meepleai -c "VACUUM FULL ANALYZE;"`	During low traffic
Performance review	Check Grafana dashboards	Compare month-over-month
Clean Docker resources	`docker system prune -a -f`	Reclaim disk space

Quarterly Checklist

Task	Notes
Secret rotation	Follow schedule in Section 3
DR test	Full restore to test environment
Capacity review	Check scaling tier adequacy
Security review	Dependency audit, vulnerability scan
Documentation review	Update this manual if needed

Backup Summary

Data	Method	Schedule	Retention	Recovery Time
PostgreSQL	`pg_dumpall`	Daily 3 AM	7 days	30-60 min
Redis	BGSAVE + copy	Weekly	4 weeks	< 5 min (or skip)

Qdrant	Snapshot API	Weekly	30 days	15-30 min
pdf_uploads	Volume tar.gz	Weekly	30 days	10-20 min
Grafana data	Volume tar.gz	Monthly	90 days	5 min
Secrets	Encrypted vault	On change	Permanent	5 min
MinIO data	Volume tar.gz	Weekly	30 days	10-20 min
ML models	Re-download	N/A	N/A	5-15 min

Automated Backup Script

```bash
#!/bin/bash
# /opt/meepleai/backup.sh
# Run via cron: 0 3 * * * /opt/meepleai/backup.sh >> /var/log/meepleai-backup.log 2>&1

set -euo pipefail
BACKUP_DIR="/backups/meepleai/$(date +%Y%m%d)"
mkdir -p "$BACKUP_DIR"

echo "=== MeepleAI Backup $(date) ==="

# PostgreSQL
echo "Backing up PostgreSQL..."
docker exec meepleai-postgres pg_dumpall -U meepleai | gzip > "$BACKUP_DIR/postgres.sql.gz"
echo "PostgreSQL: $(du -sh "$BACKUP_DIR/postgres.sql.gz" | cut -f1)"

# Redis
echo "Backing up Redis..."
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" BGSAVE
sleep 5
docker cp meepleai-redis:/data/dump.rdb "$BACKUP_DIR/redis.rdb"
echo "Redis: $(du -sh "$BACKUP_DIR/redis.rdb" | cut -f1)"

# Qdrant
echo "Backing up Qdrant..."
curl -s -X POST "http://localhost:6333/collections/meepleai_documents/snapshots" \
  -H "api-key: $QDRANT_API_KEY" > "$BACKUP_DIR/qdrant_snapshot_response.json"
echo "Qdrant snapshot created."

# PDF uploads
echo "Backing up PDF uploads..."
docker run --rm -v meepleai_pdf_uploads:/data -v "$BACKUP_DIR":/backup alpine \
  tar czf /backup/pdf_uploads.tar.gz -C /data .
echo "PDF uploads: $(du -sh "$BACKUP_DIR/pdf_uploads.tar.gz" | cut -f1)"

# Cleanup old backups (7-day retention for daily, 30-day for weekly)
find /backups/meepleai/ -maxdepth 1 -type d -mtime +7 -exec rm -rf {} \;
```

```
echo "=== Backup complete ==="
```

Disaster Recovery Procedures

Recovery Targets

Metric	Target
RTO (Recovery Time Objective)	< 2 hours
RPO (Recovery Point Objective)	< 24 hours

Scenario 1: VPS Failure (Total Loss)

RTO: 1-2 hours | **RPO**: Last backup (< 24h)

```
# 1. Provision new VPS (Hetzner Cloud)
# - Same or larger tier
# - Ubuntu 22.04+, Docker installed

# 2. Clone repository
git clone <repo-url> /opt/meepleai
cd /opt/meepleai

# 3. Restore secrets from encrypted vault
# Copy all .secret files to infra/secrets/
# OR regenerate with: pwsh infra/secrets/setup-secrets.ps1 -SaveGenerated

# 4. Start infrastructure (DB + cache)
cd infra && docker compose up -d postgres redis qdrant

# 5. Wait for health
sleep 30
docker compose ps

# 6. Restore PostgreSQL from backup
gunzip -c /backups/postgres.sql.gz | docker exec -i meepleai-postgres psql -U meepleai

# 7. Restore PDF uploads
docker run --rm -v meepleai_pdf_uploads:/data -v /backups:/backup alpine \
  sh -c "cd /data && tar xzf /backup/pdf_uploads.tar.gz"

# 8. Start application services
docker compose up -d api web

# 9. Start AI services (if needed)
docker compose --profile ai up -d
```

```
# 10. Verify
curl http://localhost:8080/
curl http://localhost:3000/

# 11. Restore Qdrant (or re-embed)
# Option A: Restore snapshot
# Option B: Re-run embedding pipeline (preferred — ensures fresh vectors)

# 12. Restore monitoring
docker compose --profile monitoring up -d
```

Scenario 2: Database Corruption

RTO: 30-60 minutes | **RPO**: Last backup (< 24h)

```
# 1. Stop all services that write to DB
docker compose stop api web n8n orchestration-service

# 2. Drop and recreate database
docker exec meepleai-postgres psql -U meepleai -c "DROP DATABASE IF EXISTS meepleai;"
docker exec meepleai-postgres psql -U meepleai -c "CREATE DATABASE meepleai;"

# 3. Restore from backup
gunzip -c /backups/meepleai/latest/postgres.sql.gz | \
  docker exec -i meepleai-postgres psql -U meepleai

# 4. Run pending EF Core migrations
cd apps/api/src/Api && dotnet ef database update

# 5. Restart services
cd infra && docker compose up -d api web

# 6. Verify data integrity
docker exec meepleai-postgres psql -U meepleai -c \
  "SELECT 'users' AS tbl, count(*) FROM \"Administration\".\"Users\"
   UNION ALL SELECT 'games', count(*) FROM \"GameManagement\".\"Games\";"
```

Scenario 3: Ransomware / Compromised System

RTO: 2-4 hours | **RPO**: Last clean backup

```
# 1. ISOLATE immediately
# - Disconnect from network / firewall all ports
# - DO NOT restart services
```

```
# 2. Provision fresh VPS (do NOT reuse compromised server)

# 3. Rotate ALL secrets
cd infra/secrets && pwsh setup-secrets.ps1 -SaveGenerated
# Also rotate: OAuth tokens, API keys, admin password

# 4. Restore from last known-clean backup
# Follow Scenario 1 procedure

# 5. Audit
# - Review access logs
# - Check for unauthorized DB changes
# - Scan for persistence mechanisms

# 6. Harden
# - Update all packages
# - Enable firewall (ufw)
# - Review SSH keys
# - Enable 2FA on hosting provider
```

Scenario 4: Accidental Deletion

RTO: 30-60 minutes | **RPO**: Depends on what was deleted

```
# Deleted Docker volumes
# → Restore from backup (see Scenario 2)

# Deleted containers (but volumes intact)
docker compose up -d  # Recreates containers with existing volumes

# Deleted specific data (e.g., a user, a game)
# → Restore DB from backup to temp container, extract needed records
docker run -d --name temp-postgres -e POSTGRES_PASSWORD=temp pgvector/pgvector:pg16
gunzip -c backup.sql.gz | docker exec -i temp-postgres psql -U postgres
# Query temp DB, manually insert missing records into production
docker rm -f temp-postgres

# Deleted code / git
git reflog  # Find lost commits
git reset --hard <commit-hash>  # Restore to known state
```

Infrastructure Hardening Checklist

Area	Action	Frequency
Firewall	Only expose 80, 443 (via Traefik)	On deploy

SSH	Key-only auth, disable root login	On deploy
Docker	No `--privileged`, use `no-new-privileges`	On deploy
Secrets	File-based, never in environment blocks	Always
Updates	`apt update && apt upgrade`	Monthly
Monitoring	All services have health checks	Always
Backups	Automated daily, tested monthly	Always
TLS	Traefik with Let's Encrypt auto-renewal	On deploy

Resource Monitoring Thresholds

Metric	Warning	Critical	Action
CPU	> 70% sustained	> 85% sustained	Scale up VPS tier
Memory	> 75% used	> 90% used	Increase RAM or reduce services
Disk	< 30% free	< 15% free	Prune Docker, archive old backups
Swap	> 50% used	> 80% used	Add RAM, reduce service count
Network	> 80% bandwidth	> 95% bandwidth	Check for abuse, scale up

Cost Summary

Component	Monthly Cost	Notes
Hetzner VPS (CPX41)	EUR 29.52	8 vCPU, 16 GB RAM
Domain	~EUR 1	Annual / 12
Backups (Hetzner)	EUR 5.90	20% of VPS cost
S3 storage (R2)	~EUR 0-5	$0.015/GB, free egress
Total	**~EUR 36-41/mo**	

For beta (CPX31): ~EUR 22/mo. For full production (CPX51): ~EUR 72/mo.

Appendix A: Complete Command Reference

Service Management

```
# Start/stop
docker compose up -d                                    # Start core
docker compose --profile ai up -d                       # Start AI services
docker compose --profile monitoring up -d               # Start monitoring
docker compose --profile automation up -d               # Start automation
docker compose --profile storage up -d                  # Start MinIO
docker compose down                                     # Stop all
docker compose down -v                                  # Stop + delete volumes (DANGER)

# Status
docker compose ps                                       # Service status
docker stats --no-stream                                # Resource usage
docker system df                                        # Disk usage

# Logs
docker logs <container> --tail=50                       # Recent logs
docker logs <container> -f                              # Follow logs
docker compose logs --tail=100 --since=5m               # All services, last 5 min

# Maintenance
docker compose restart <service>                        # Restart one
docker compose up -d --build <service>                  # Rebuild + restart
docker compose pull                                     # Pull latest images
docker system prune -a -f                               # Clean everything unused
```

PostgreSQL

```
docker exec meepleai-postgres pg_isready -U meepleai
docker exec meepleai-postgres psql -U meepleai -c "<SQL>"
docker exec meepleai-postgres pg_dumpall -U meepleai | gzip > backup.sql.gz
gunzip -c backup.sql.gz | docker exec -i meepleai-postgres psql -U meepleai
```

Redis

```
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" ping
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" INFO stats
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" --bigkeys
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" BGSAVE
docker exec meepleai-redis redis-cli -a "$REDIS_PASSWORD" FLUSHALL    # DANGER
```

Qdrant

```
curl http://localhost:6333/healthz
curl http://localhost:6333/collections -H "api-key: $QDRANT_API_KEY"
curl http://localhost:6333/collections/meepleai_documents -H "api-key: $QDRANT_API_KEY"
curl -X POST "http://localhost:6333/snapshots" -H "api-key: $QDRANT_API_KEY"
```

EF Core Migrations

```
cd apps/api/src/Api
dotnet ef migrations add <Name>
dotnet ef database update
dotnet ef migrations list
dotnet ef migrations script <From> <To>
```

Secrets

```
cd infra/secrets
pwsh setup-secrets.ps1 -SaveGenerated      # Initial setup
pwsh setup-secrets.ps1 -Validate            # Validate all present
```

Appendix B: Environment-Specific Notes

Development (compose.dev.yml)

- All ports exposed to host
- Volume: `postgres_dev_data` (separate from prod)
- Hot reload enabled for web service
- Mailpit as SMTP trap
- MinIO for local S3 testing

Staging (compose.staging.yml)

- Traefik reverse proxy (compose.traefik.yml)
- Let's Encrypt staging certificates
- Separate volumes (5 staging-specific)
- Reduced resource limits

Production (compose.prod.yml)

- Traefik with production Let's Encrypt

- Separate volumes (5 prod-specific)
- Full resource limits
- `traefik_certs` volume for certificate persistence
- No development tools (Mailpit, etc.)

Integration Tests (compose.integration.yml)

- Ephemeral containers for CI
- Test database (separate from dev/prod)
- Minimal resource allocation

Appendix C: Runbook Cross-Reference

Existing runbooks in `docs/operations/runbooks/`:

Runbook	When to Use
`ai-quality-low.md`	RAG quality below acceptable thresholds
`dependency-down.md`	External dependency (BGG, OpenRouter) unavailable
`error-spike.md`	Sudden increase in error rate
`high-error-rate.md`	Sustained high error rate (> 5%)
`infrastructure-monitoring.md`	Monitoring infrastructure issues
`slow-performance.md`	API latency exceeding thresholds

Appendix D: Key File Paths

File	Purpose
`infra/docker-compose.yml`	Base compose (environment-neutral)
`infra/compose.dev.yml`	Development overrides
`infra/compose.prod.yml`	Production overrides
`infra/compose.staging.yml`	Staging overrides
`infra/compose.traefik.yml`	Traefik reverse proxy overlay
`infra/compose.integration.yml`	CI test overrides
`infra/prometheus.yml`	Prometheus scrape config

`infra/prometheus-rules.yml`	Alert rules
`infra/alertmanager.yml`	Alert routing config
`infra/grafana-datasources.yml`	Grafana data source provisioning
`infra/grafana-dashboards.yml`	Grafana dashboard provisioning
`infra/dashboards/`	Grafana dashboard JSON files
`infra/redis/redis.conf`	Redis configuration
`infra/init/postgres-init.sql`	PostgreSQL initialization SQL
`infra/secrets/`	Secret files directory
`infra/secrets/setup-secrets.ps1`	Secret setup script
`infra/scripts/load-secrets-env.sh`	Secret loading entrypoint
`infra/Makefile`	Make targets for common operations
`infra/start-*.sh` / `infra/start-*.ps1`	Start helper scripts
`infra/stop.ps1`	Stop helper script